

idTech4 RenderBump

Blender 3.2+ Normal Map Baking Add-on

Version 1.0.0 · Replicated from the Doom 3 GPL source

1. Overview

The idTech4 RenderBump add-on is a bit-for-bit re-implementation of the renderbump console command from the Doom 3 / Quake 4 / Prey engine, including the exact model-loading path the engine takes to get there. Given the same low- and high-poly models the engine was given, it produces the same tangent-space normal map, stored as an internal Blender image.

IMPORTANT: This is a REPLICATION, not an improvement. Every quirk of the original is kept on purpose: the lazy 100-step ray march through the triangle hash, the epsilon in CreateTriHash that only ever decrements the lower bin (and the typo that applies the second epsilon to the wrong array), the farthest-hit-wins trace, the dupVerts normal merge that loses contributions on three-way shared vertices, and the truncating — not rounding — float to byte conversion. If you want a better normal map, use Blender's own baker. If you want the map the engine would have produced, use this.

Use it when you need a normal map that matches a shipped idTech4 asset, when you are rebuilding an asset whose renderbump output must drop into an existing game and look identical, or when you want to understand why the shipped map looks the way it does.

2. Installation

1. Open Blender and navigate to Edit > Preferences > Add-ons.
2. Click Install... and select idTech4_renderbump.py.
3. Tick the checkbox next to "idTech4 RenderBump".
4. Close Preferences. The add-on is now active.

The add-on registers a single panel: 3D Viewport > N-Panel > idTech4 Renderbump > RenderBump. It adds no File menu entries and needs no shared source directories — it is the one add-on in the toolchain with no connection to the .mtr tree.

3. Three Things Worth Knowing First

3.1 The Low Poly's Normals Are Ignored

The low-poly model's normals and tangents are NOT taken from the file. The engine throws away any explicit normals on a surface whose material carries a renderBump command — "completely ignore any explicit normals on surfaces with a renderbump command, which will guarantee the best contours and least vertexes" — and regenerates them. So does this. The smooth shading you see in Blender on the low poly has no effect at all on the result.

3.2 The High Poly's Normals Depend on the Format

The high-poly model's normals ARE taken from the file, but where they come from depends on which format it is.

Format	Where the normals come from
.lwo	A .lwo carries no usable normals at all. The engine recomputes LightWave's own smoothing: each polygon's flat normal, plus every adjacent polygon that shares the point, is in the same smoothing group and sits within the surface's SMAN angle. That is not the rule Blender's shade-auto-smooth uses and not what Blender's quantised custom split normals hold, so this add-on recomputes it too. Set High Smoothing Angle to the source surface's SMAN value — LightWave's default, and what the shipped Doom 3 models use, is 90°.
.ase	The other way around: an .ase stores finished per-corner normals in *MESH_NORMALS and the engine uses them verbatim. The smoothing angle does not apply and the control is hidden.

3.3 Read the Files, Not the Scene

Both formats can be read straight off disk, and that is the accurate way to use this. Going through Blender costs you something either way: the .lwo importer welds points that round together, and the .ase importer deliberately undoes the winding and V flips the engine applies at parse time. Reading the file skips all of it.

idTech 4 games do not ship their assets as loose files. Doom 3, Quake 4, Prey and The Dark Mod pack them into .pk4 archives; Doom 3 BFG packs them into .resources files. This add-on does not read inside either. It reads an ordinary directory tree on disk, and nothing else.

IMPORTANT: Extract the game's archives before pointing this add-on at anything inside them. A .pk4 is an ordinary zip archive that any zip tool will open; a .resources file needs a Doom 3 BFG extraction tool. Extract every archive into one folder, preserving the paths inside them, so you end up with the game's real directory structure.

For this add-on that means the two model files themselves, and the .mtr whose renderBump line records the size and anti-alias the shipped normal map was built with. Extracting the game's models/ and materials/ trees is enough; there is no Base Directory to configure here.

NOTE: When the high poly comes from a scene object, the add-on says so in its own log and reminds you that the .lwo importer welds points the engine does not — the engine loads a renderbump high poly with fastLoad, where the vertex remap is the identity. If the file and the scene object disagree, trust the file.

4. The RenderBump Panel

3D Viewport > N-Panel > idTech4 Renderbump tab. Pick a low poly, pick a high poly, set the size, and press RenderBump.

4.1 Low Poly (the UV layout)

Option	Type	Default	Description
Low Poly From	<i>Enum</i>	Scene Object	Scene Object or Model File — see Section 3.3.
Low Poly	<i>Object</i>	—	The low-poly mesh object. Its UV layout is what gets baked into. Shown when the source is Scene Object.
Low Poly File	<i>Path</i>	—	A .lwo or .ase read straight off disk. Shown when the source is Model File.

4.2 High Poly (the detail)

Option	Type	Default	Description
High Poly From	<i>Enum</i>	Scene Object	Scene Object or Model File.
High Poly	<i>Object</i>	—	The high-poly mesh object. Shown when the source is Scene Object.
High Poly File	<i>Path</i>	—	A .lwo or .ase read straight off disk.
High Smoothing Angle	<i>Float</i>	90°	The high-poly surface's LightWave SMAN smoothing angle. Only used when the high poly comes from a scene object; a .lwo carries its own and an .ase carries finished normals instead. Shown only for a scene object.

4.3 Bake Settings

These correspond one-for-one to the arguments of the engine's own renderbump command.

Option	Type	Default	Description
Width / Height	<i>Int</i>	512 × 512	renderbump -size <width> <height>. Range 1 – 8192 each.
Trace	<i>Float</i>	0.05	renderbump -trace: how far a ray may travel, as a fraction of the high poly's longest bounding axis. Range 0.001 – 1.0.
Anti-Alias	<i>Int</i>	1	renderbump -aa: render at 2^n the requested size and mip it back down. Range 0 – 3.
Outline	<i>Int</i>	8	renderbump -outline: pixels of bleed added around every filled region, for bilinear filtering and mip-mapping. Range 0 – 64.

4.4 Output and Accuracy Settings

Option	Type	Default	Description
Use World Transform	<i>Bool</i>	On	Bake in each object's world matrix, so a low and high poly placed differently in the scene still line up. Only meaningful for scene objects.
Global Map	<i>Bool</i>	Off	renderbump globalMap: also emit the object-space normal map, as <name>_global.
Color Map	<i>Bool</i>	Off	renderbump colorMap: also emit the sampled vertex colours, as <name>_color.
SIMD	<i>Enum</i>	SSE (exact)	Which build of the engine to match — see Section 5.
Image Name	<i>String</i>	—	Name for the generated image. Blank uses the low poly's name plus _local.

Results are written into internal Blender image datablocks, with colorspace set to Non-Color: a normal map is data, and Non-Color also makes the float-to-byte round trip exact. The rows are flipped on the way in, because Blender's pixel buffer starts at the bottom row and the engine's starts at the top.

The operator prints a running log to the system console and reports which images it wrote, at what size, when it finishes. Warnings it may print include non-triangle polygons in either model — the engine skips those outright with "Make sure you triplet it down", so they contribute nothing.

5. Accuracy

Measured against the reference gizmo1_local.tga shipped with the test pair, built from the same two .lwo files at -size 1024 1024 -aa 0 (not the -size 128 128 -aa 2 the shipped mapobjects.mtr asks for — the reference was rebaked at the larger size):

Measure	Result
Bit-identical texels	99.25%
Texels within one 8-bit step	99.996%
Traced / untraced coverage mask agreement	all but one texel in a million

The residue is sub-LSB floating point noise. The original is a 32-bit x86 build whose tangent and plane builders normalise with the SSE rsqrtps instruction, a hardware approximation accurate to about $1.5e-4$ whose exact result is a property of the CPU's internal table and is not reproducible in software. 99.25% is therefore the ceiling, not a defect to be chased.

SIMD setting	Matches
SSE (exact) — default	A stock 32-bit Doom 3 build, whose tangent and plane builders normalise with SSE rsqrtps. Scores 99.25% on the reference pair.
Generic SIMD (RSqrt)	A build with the SIMD assembly compiled out, which falls back to idMath::RSqrt — dhewm3, for instance. Scores 95.9% against a stock build's output, which is correct: it is matching a different build.

NOTE: Choose SSE (exact) unless you are specifically reproducing the output of a build with the SIMD assembly disabled.

6. Which Engine Code This Mirrors

Every step is written against the Doom 3 GPL source, function by function, so the two can be diffed by eye.

Source file	What is replicated
renderer/Model_lwo.cpp	lwGetPolyNormals / lwGetVertNormals / lwGetPointPolygons — the LWO smoothing pass.
renderer/Model_ase.cpp	ASE_Parse and every ASE_Key* callback under it, including the two bugs in ASE_KeyMESH_CVERTLIST.
framework/FileSystem.cpp	OSPathToRelativePath, which is what turns an .ase *BITMAP into a material name.
renderer/Model.cpp	ConvertLWOToModelSurfaces and ConvertASEToModelSurfaces, each in both the normal path (the low poly, via CheckModel) and the fastLoad path (the high poly, via PartialInitFromFile).
renderer/tr_trisurf.cpp	R_CleanupTriangles and everything under it: CreateSillIndexes, RemoveDegenerateTriangles, DuplicateMirroredVertexes, CreateDupVerts, DeriveFacePlanes, DeriveTangents.
idlib/math/Simd_Generic.cpp	DeriveTriPlanes / DeriveTangents / NormalizeTangents.

Source file	What is replicated
tools/compiler/renderbump/ renderbump.cpp	CreateTriHash, TraceToMeshFace, SampleHighMesh, RasterizeTriangle, RenderBumpTriangles, OutlineNormalMap, WriteRenderBump.

Floating point is handled to match as well: every expression the original evaluates in float is evaluated in float here, and anything the original evaluates on the x87 stack — a 32-bit MSVC build promotes float expressions to 80-bit and only rounds on store — is evaluated in float64 and rounded once, at the store.

7. Tips and Troubleshooting

The Result Has Holes or Missing Regions

Raise Trace. It is a fraction of the high poly's longest bounding axis, so a model with one long dimension needs a larger value than its proportions suggest. Remember also that the trace is farthest-hit-wins, which is the engine's behaviour and can pick a surface you did not expect when the two models are far apart.

Seams Along UV Islands

Raise Outline. That is exactly what it is for — bleed around every filled region so bilinear filtering and mip-mapping do not sample empty texels.

The High Poly Looks Faceted Where It Should Be Smooth

A .lwo high poly read from a scene object is being shaded with the wrong smoothing angle. Set High Smoothing Angle to the source surface's SMAN value — 90° for the shipped Doom 3 models — or better, point the add-on at the .lwo file directly, which carries its own.

The Result Does Not Match a Shipped Normal Map

Check the size and anti-alias values first: the shipped .mtr's renderBump line records the arguments the asset was actually built with, and they are frequently much smaller than you would choose today. Then read the two sources from file rather than from the scene, and leave SIMD on SSE (exact).

Non-Triangle Polygons

Both models are reported if they contain any. The engine skips them outright, so they contribute nothing to the bake — triangulate before baking if that is not what you want.

It Is Slow

It is a faithful software replication of a bake the engine did on the CPU, and the ray march is deliberately the engine's lazy 100-step version. Bake at a smaller size while iterating, then do the final run at full size with the anti-alias value the material asks for. A progress bar runs throughout.

idTech4 RenderBump — end of documentation